Data 252: Classification Project

Siera Edwards, Haydn Duncan, Mamadou Dem, Bradley Allen

2025-04-30

Contents

Wrangle data **4**

Pairs Plot **5**

Scatter Plots **7**

Full data set: Temperature and absolute magnitude 8

Full data set: Temperature and star color 9

kNN Models **10**

Variables Used 10

First kNN Model 11

Build Model 11

Confusion Matrix 11

Accuracy 11

Error 11

Select Best k Value 12

Pick a neighborhood 12

Plot CPs 12

Final kNN Model 13

Confusion Matrix 13

Accuracy 13

Error 13

Tree Models **14**

Variables used 14

Single Classification Tree 15

Build Model 15

Plot Tree 15

Plot CP . . . 16
Print CP . . . 16
Min CP . . . 17
Pruned Tree . . . 17
Predict w/ Tree Models . . . 18
Default Tree . . . 18
Pruned Tree . . . 18
Random Forest . . . 19
Make Model . . . 19
Best tuning parameter . . . 19
Final Model . . . 19
Confusion Matrix . . . 20
Correct Rate . . . 20
Variables by Importance . . . 20

Bagging Models **21**
Variables Used . . . 21
Default Bagging Model . . . 22
Build Model . . . 22
Confusion Matrix . . . 22
Correct Rate . . . 22
Caret Bagging . . . 23
Build Model . . . 23
Confusion Matrix . . . 23
Correct Rate . . . 23
Boosting . . . 24
Model . . . 24
Best Tuning Parameter . . . 24
Confusion Matrix . . . 24
Correct Rate . . . 24

Regression **25**
Variables Used . . . 25
Simple Linear Regression . . . 26
Plot . . . 26
Logistic Regression . . . 27
Model . . . 27

Plot . . . 27
Confusion Matrix . . . 28
Correct Rate . . . 28
Optimal Logistic Regression Model . . . 29
Variable Selection . . . 29
Backward Selection . . . 30
Forward Selection . . . 31
Best Subset . . . 32
Best Model . . . 32
Plot Best Model . . . 33
Predict w/ Best Model . . . 33
Confusion Matrix . . . 33
Correct Rate . . . 33
Compare Models . . . 34
Simple Logistic . . . 34
Optimal Logistic Model . . . 34

Discriminant Analysis **35**

LDA . . . 36
Make Model . . . 36
Plot . . . 36
Correct Rate . . . 37
Partition Plot . . . 37
QDA . . . 38
Make Model . . . 38
Correct Rate . . . 38
Partition Plot . . . 39

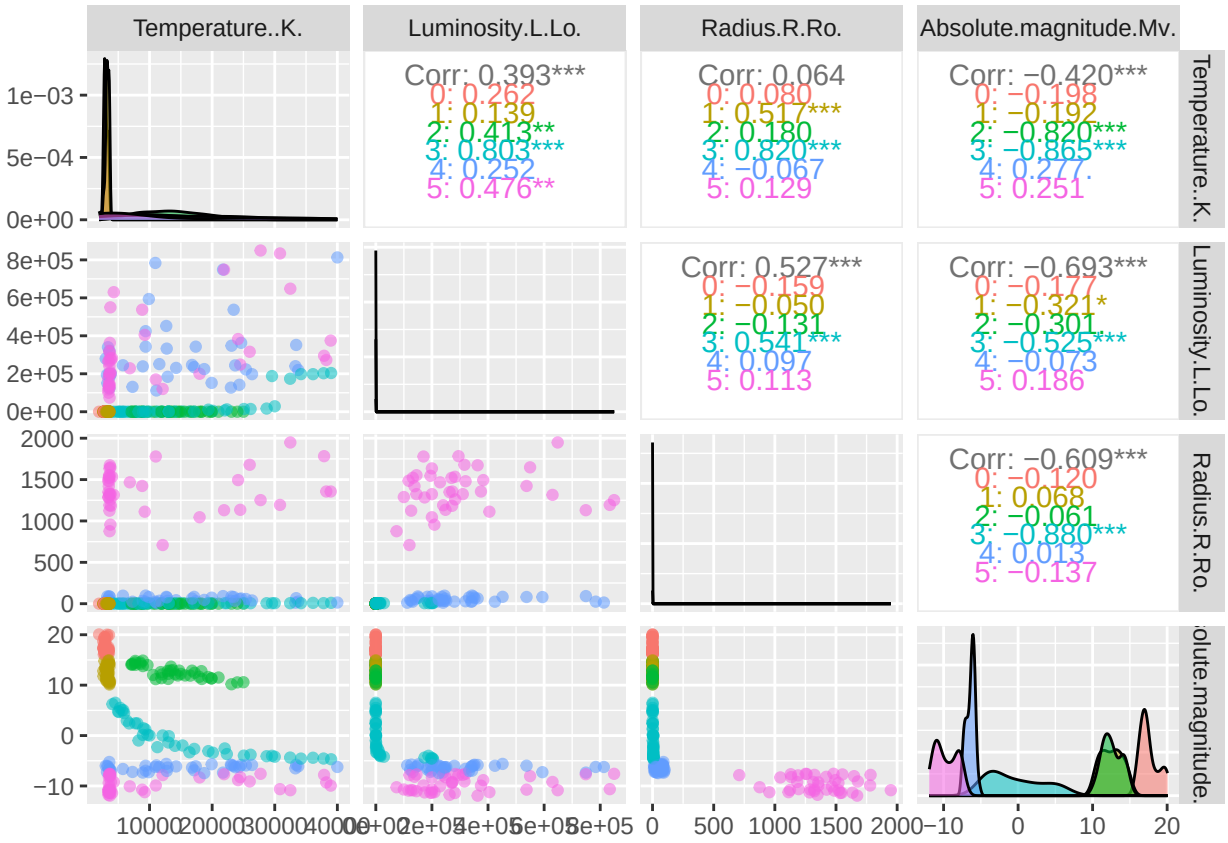
Wrangle data

```
# Dataset: https://www.kaggle.com/datasets/deepu1109/star-dataset

# READ IN DATA
stars = read.csv("C:/Users/bradl/OneDrive/Willamette/2025_01_Spring/Data_252/Group_Project_Classificati
#stars = read.csv("/Users/haydnduncan/Desktop/DATASCI/MML Class (R Undergrad)/Classification Project/6
#stars = read_csv("6 class csv.csv")
```

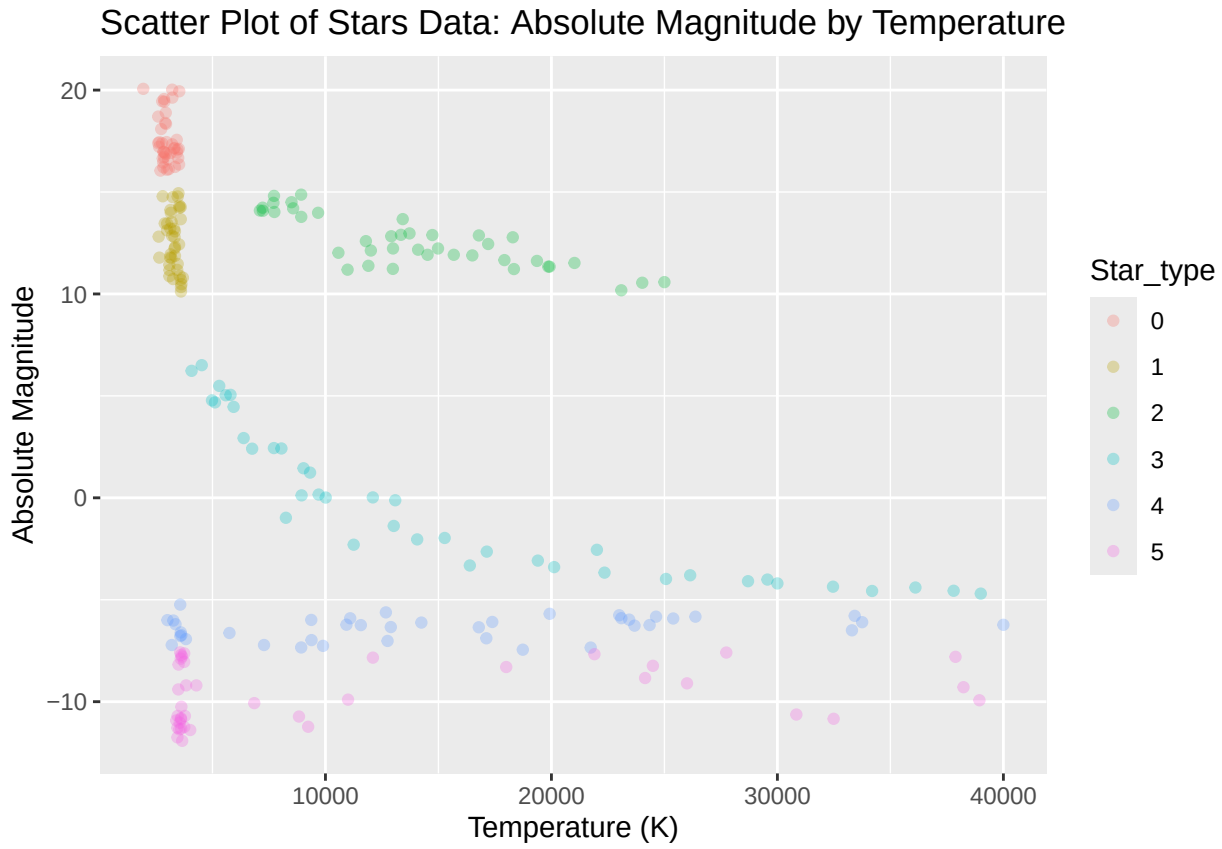
```
# WRANGLE DATA
stars = mutate(stars, Star.color = as.factor(Star.color))
stars$Star.color = fct_recode(stars$Star.color,
            "Blue" = "Blue ",
            "Blue white" = "Blue-white",
            "Blue white" = "Blue-White",
            "Blue white" = "Blue white",
            "Blue white" = "Blue White",
            "Blue white" = "Blue white",
            "Blue white" = "Blue white",
            "Blue white" = "Blue white ",
            "Orange" = "Orange-Red",
            "Orange" = "Pale yellow orange",
            "White" = "white",
            "White" = "White-Yellow",
            "White" = "Whitish",
            "Yellowish" = "yellow-white",
            "Yellowish" ="yellowish",
            "Yellowish" ="Yellowish White")
stars = stars %>% mutate(Star_type = as.factor(stars$Star.type))
stars = na.omit(stars)
stars2 = dplyr::select(stars,
                -Star.type,
                -Spectral.Class,
                -Absolute.magnitude.Mv.,
                -Radius.R.Ro.,
                -Luminosity.L.Lo.)
```

Pairs Plot


Temperature..K.
Luminosity.L.Lo.
Radius.R.Ro.
Absolute.magnitude.Mv.
Corr: 0.393***
0: 0.262
1: 0.139
2: 0.413**
3: 0.803***
4: 0.252
5: 0.476**
Corr: 0.064
0: 0.080
1: 0.517***
2: 0.180
3: 0.820***
4: −0.067
5: 0.129
Corr: −0.420***
0: −0.198
1: −0.192
2: −0.820***
3: −0.865***
4: 0.277.
5: 0.251
Corr: 0.527***
0: −0.159
1: −0.050
2: −0.131
3: 0.541***
4: 0.097
5: 0.113
Corr: −0.693***
0: −0.177
1: −0.321*
2: −0.301
3: −0.525***
4: −0.073
5: 0.186
Corr: −0.609***
0: −0.120
1: 0.068
2: −0.061
3: −0.880***
4: 0.013
5: −0.137
1e−03
5e−04
0e+00
8e+05
6e+05
4e+05
2e+05
0e+00
2000
1500
1000
500
0
20
10
0
−10
10000
20000
30000
40000
0
500
1000
1500
2000
−10
0
10
20

Scatter Plots

Full data set: Temperature and absolute magnitude

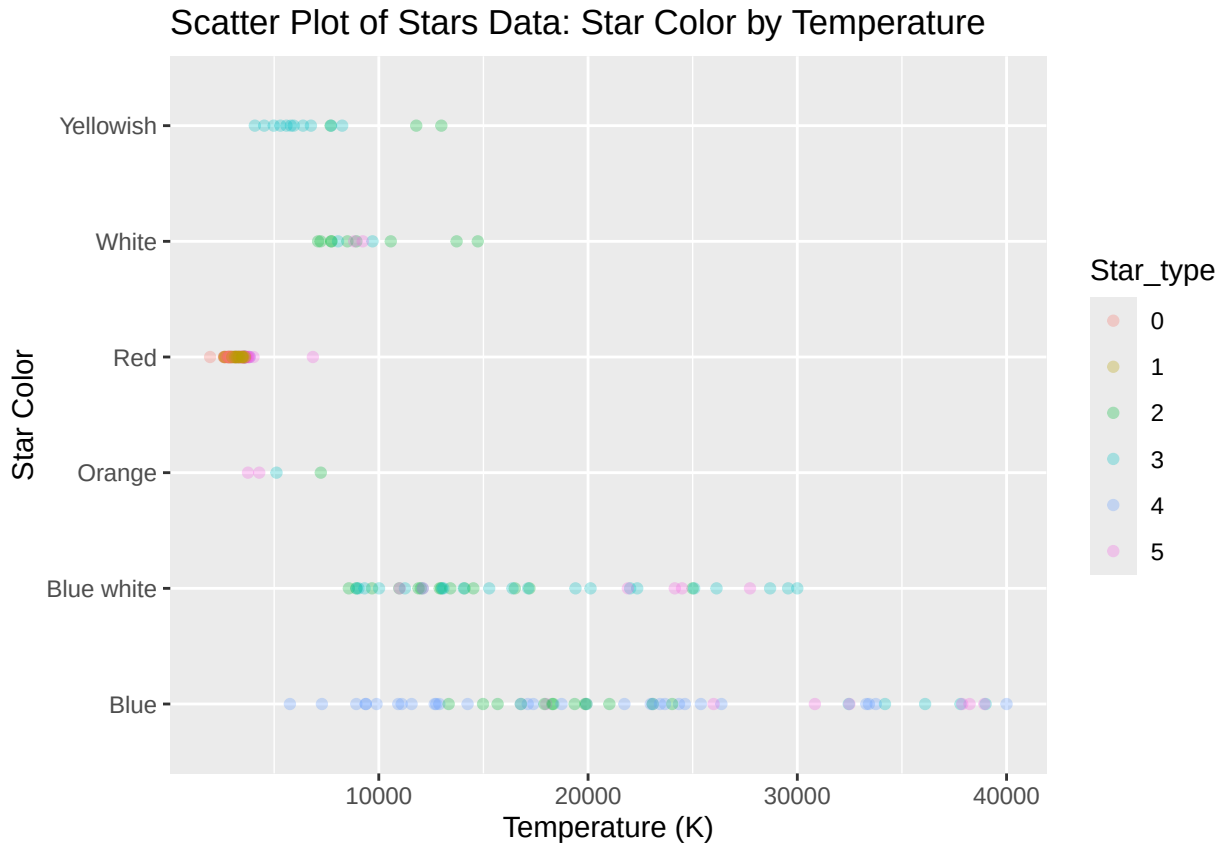


With absolute magnitude included, the data points were grouped such that most models predicted with 100% accuracy.

We ended up removing variables to create noise so that the models could be properly applied / tested.

Updated plot on next page.

Full data set: Temperature and star color



Plot with absolute magnitude and radius removed. Used color as the response.

This grouped the points such that there was enough noise to test the models.

kNN Models

Variables Used

```
## 'data.frame':    168 obs. of  4 variables:
##  $ Temperature..K. : num  -0.749 -0.803 -0.808 -0.827 -0.827 ...
##  $ Luminosity.L.Lo.: num  -0.597 -0.597 -0.597 -0.597 -0.597 ...
##  $ Star.color      : Factor w/ 6 levels "Blue","Blue white",..: 4 4 4 4 4 4 4 4 4 4 ...
##  $ Star_type       : Factor w/ 6 levels "0","1","2","3",..: 1 1 1 1 1 1 1 1 1 1 ...
```

First kNN Model

Build Model

```
knnStars<- knn(train = star.train[,c(1,2)],
               test = star.test[,c(1,2)],
               cl = star.train$Star_type,
               k = 7)
```

Confusion Matrix

```
table(knnStars, star.test$Star_type)
```

```
##
## knnStars  0  1  2  3  4  5
##        0  7  2  0  0  0  0
##        1  5 10  0  1  0  0
##        2  0  0  9  7  0  0
##        3  0  0  3  4  0  0
##        4  0  0  0  0  6  4
##        5  0  0  0  0  6  8
```

Accuracy

```
mean(knnStars==star.test$Star_type)
```

```
## [1] 0.6111111
```

Error

```
mean(knnStars!=star.test$Star_type)
```

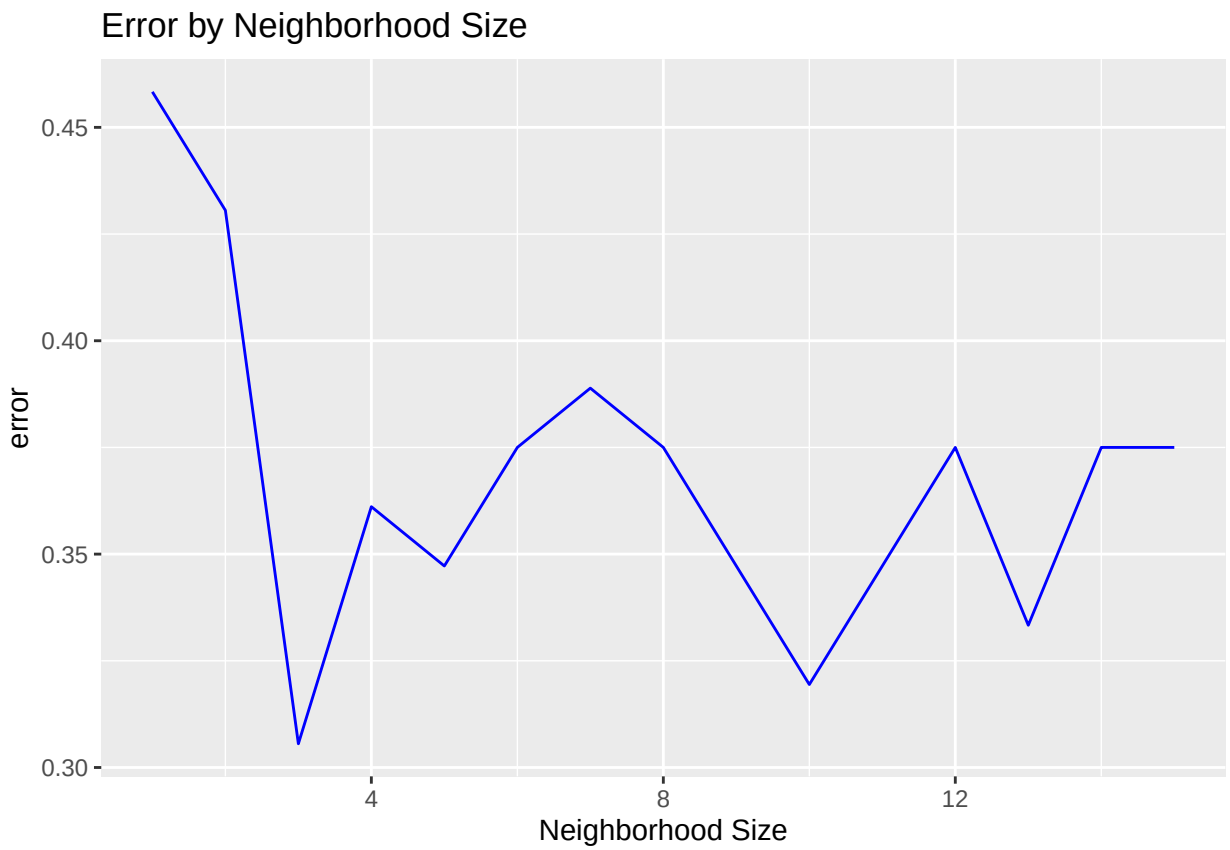
```
## [1] 0.3888889
```

Select Best k Value

Pick a neighborhood

```
error <- rep(0,15)
for (i in 1:15) {
  knnStars<- knn(train = star.train[,c(1,2)],
                 test = star.test[,c(1,2)],
                 cl = star.train$Star_type, k = i)
  error[i] = 1- mean(knnStars == star.test$Star_type)
}
```

Plot CPs



Best value appears to be 3.

Final kNN Model

```
stars_pred <- knn(train = star.train[,1:2],
                  test = star.test[,1:2],
                  cl = star.train$Star_type,
                  k=3)
```

Confusion Matrix

```
table(stars_pred, star.test$Star_type)
```

```
##
## stars_pred  0  1  2  3  4  5
##          0  8  2  0  0  0  0
##          1  4 10  0  0  0  0
##          2  0  0  7  4  0  0
##          3  0  0  5  8  0  0
##          4  0  0  0  0  7  2
##          5  0  0  0  0  5 10
```

Accuracy

```
mean(stars_pred==star.test$Star_type)
```

```
## [1] 0.6944444
```

Error

```
mean(stars_pred!=star.test$Star_type)
```

```
## [1] 0.3055556
```

Tree Models

Variables used

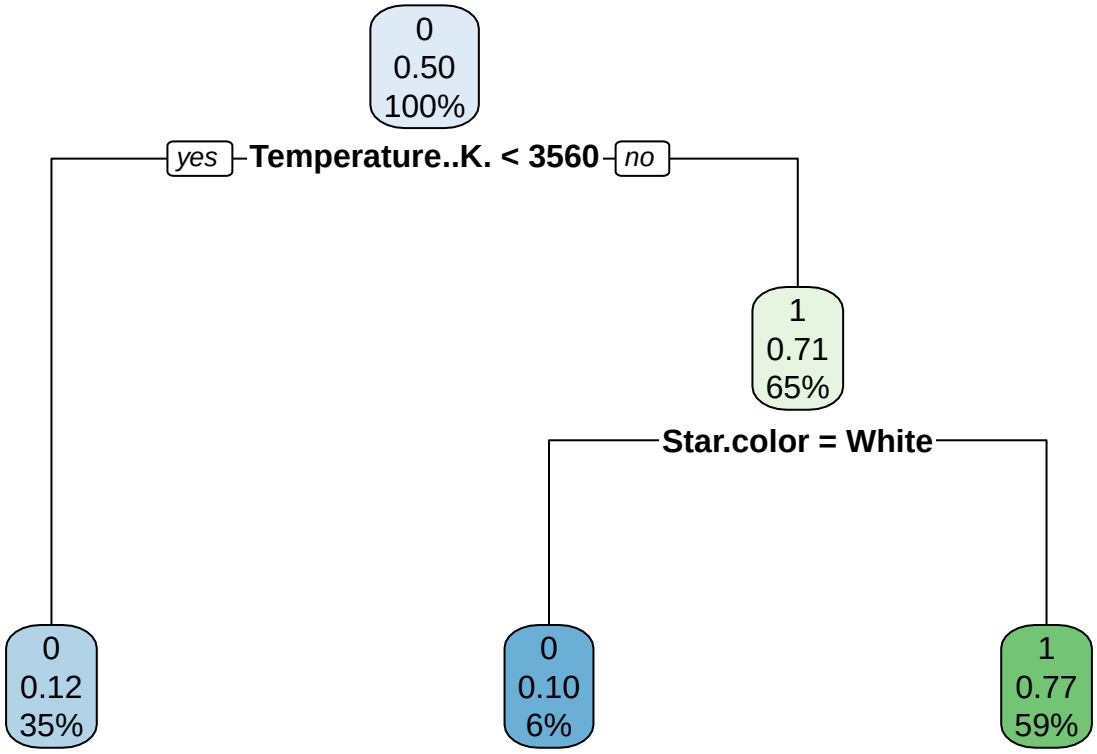
```
## 'data.frame':    240 obs. of  3 variables:
##  $ Temperature..K.: int  3068 3042 2600 2800 1939 2840 2637 2600 2650 2700 ...
##  $ Star.color     : Factor w/ 6 levels "Blue","Blue white",..: 4 4 4 4 4 4 4 4 4 4 ...
##  $ starSize       : Factor w/ 2 levels "0","1": 1 1 1 1 1 1 1 1 1 1 ...
```

Single Classification Tree

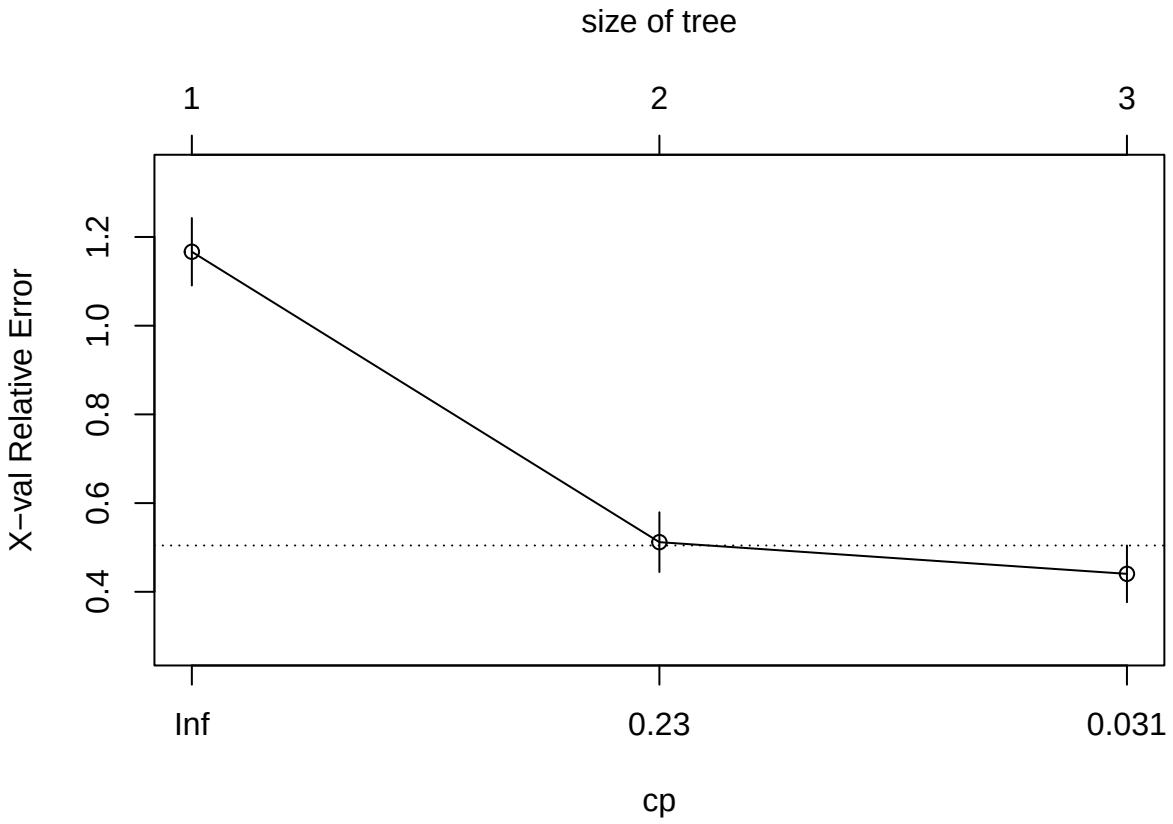
Build Model

```
set.seed(252)
classTree<- rpart(starSize ~ .,
                  data = log.trainCaret,
                  method = "class")
```

Plot Tree



Plot CP



Print CP

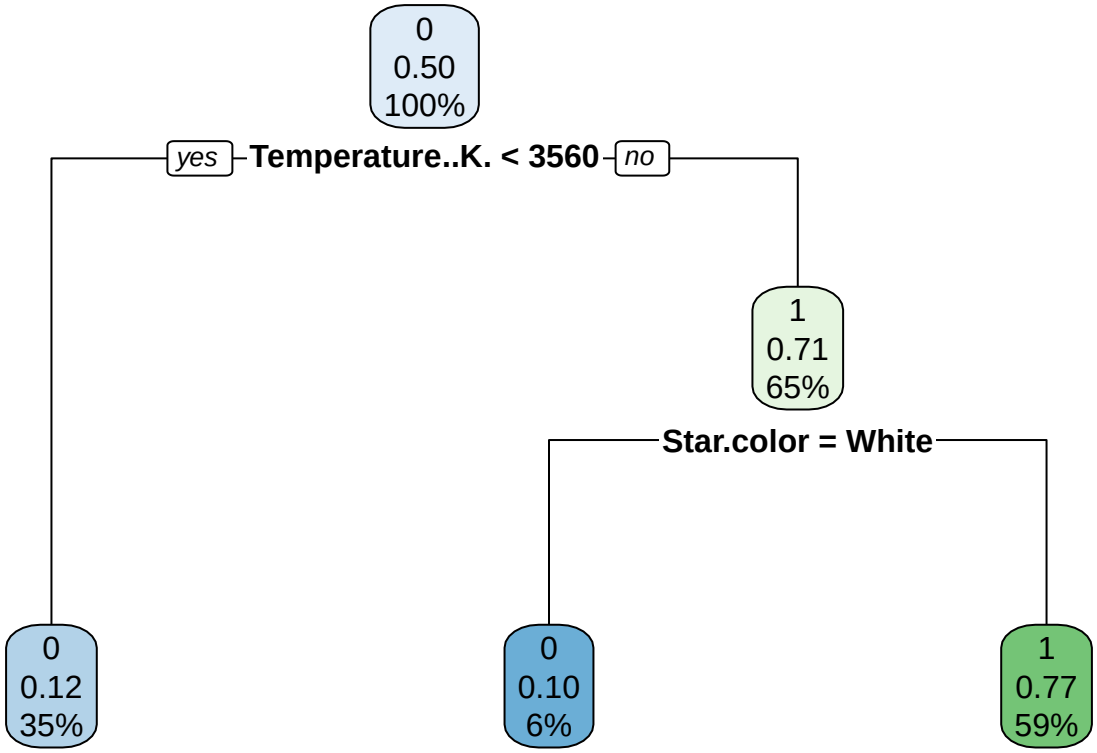
```
##
## Classification tree:
## rpart(formula = starSize ~ ., data = log.trainCaret, method = "class")
##
## Variables actually used in tree construction:
## [1] Star.color      Temperature..K.
##
## Root node error: 84/168 = 0.5
##
## n= 168
##
##         CP nsplit rel error  xerror     xstd
## 1 0.535714      0   1.00000 1.16667 0.076073
## 2 0.095238      1   0.46429 0.51190 0.067337
## 3 0.010000      2   0.36905 0.44048 0.063944
```

Min CP

```
minCP<-classTree$cptable[which.min(classTree$cptable[,"xerror"]),"CP"]
minCP
```

```
## [1] 0.01
```

Pruned Tree



Predict w/ Tree Models

Default Tree

```
cmTree1<-table(log.testCaret$starSize, predTree1)
cmTree1
```

Confusion Matrix

```
##    predTree1
##      0  1
##   0 21 15
##   1  8 28
```

```
mean(log.testCaret$starSize==predTree1)
```

Correct Rate

```
## [1] 0.6805556
```

Pruned Tree

```
cmTree2<-table(log.testCaret$starSize, predTree2)
cmTree2
```

Confusion Matrix

```
##    predTree2
##      0  1
##   0 21 15
##   1  8 28
```

```
mean(log.testCaret$starSize==predTree2)
```

Correct Rate

```
## [1] 0.6805556
```

Random Forest

Make Model

```
set.seed(252)
caretRF <- train(starSize ~.,
                 data = log.trainCaret,
                 method = "rf",
                 trControl = trainControl("cv", number = 10),
                 importance = TRUE
)
```

Best tuning parameter

```
caretRF$bestTune
```

```
##   mtry
## 1    2
```

Final Model

```
caretRF$finalModel
```

```
##
## Call:
##  randomForest(x = x, y = y, mtry = param$mtry, importance = TRUE)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 2
##
##         OOB estimate of  error rate: 20.24%
## Confusion matrix:
##    0  1 class.error
## 0 62 22   0.2619048
## 1 12 72   0.1428571
```

Confusion Matrix

```
table(predCaretRF, log.testCaret$starSize)
```

```
##
## predCaretRF  0  1
##           0 21  9
##           1 15 27
```

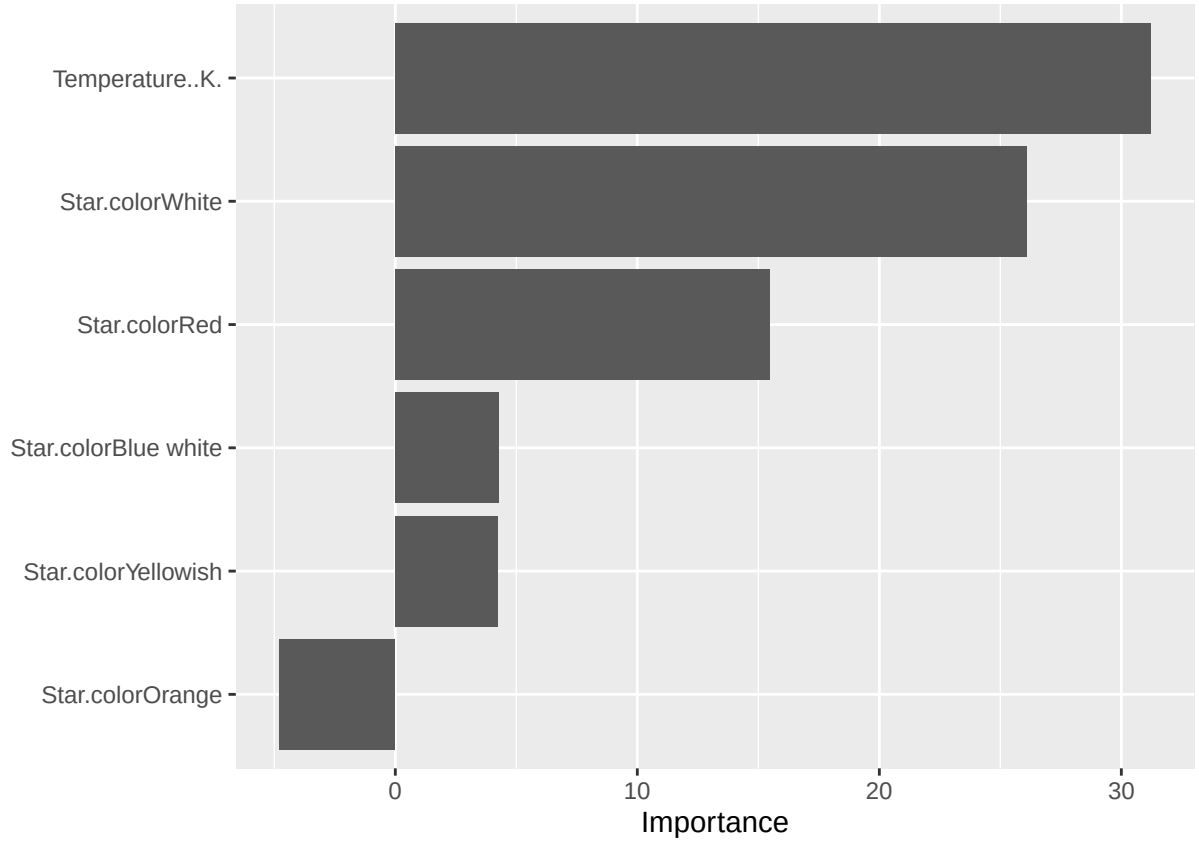
Correct Rate

```
mean(predCaretRF==log.testCaret$starSize)
```

```
## [1] 0.6666667
```

Variables by Importance

```
vip(caretRF$finalModel)
```



Bagging Models

Variables Used

```
str(bag.trainCaret)
```

```
## 'data.frame':    168 obs. of  4 variables:
##  $ Temperature..K. : int  3068 3042 2800 1939 2637 2650 2700 3600 3129 3134 ...
##  $ Luminosity.L.Lo.: num  0.0024 0.0005 0.0002 0.000138 0.00073 0.00069 0.00018 0.0029 0.0122 0.0004
##  $ Star.color      : Factor w/ 6 levels "Blue","Blue white",..: 4 4 4 4 4 4 4 4 4 4 ...
##  $ Star_type       : Factor w/ 6 levels "0","1","2","3",..: 1 1 1 1 1 1 1 2 2 2 ...
```

Default Bagging Model

Build Model

```
starBag <- bagging(Star_type ~ .,
                   data = bag.trainCaret,
                   nbagg = 150,
                   coob = TRUE,
                   control = rpart.control(minsplit = 2, cp = 0))
```

Confusion Matrix

```
cmBag<-table(bag.testCaret$Star_type, predBag)
cmBag
```

```
##    predBag
##       0  1  2  3  4  5
##   0  9  3  0  0  0  0
##   1  2 10  0  0  0  0
##   2  0  0 12  0  0  0
##   3  0  0  0 10  2  0
##   4  0  0  0  0 10  2
##   5  0  0  0  0  3  9
```

Correct Rate

```
mean(bag.testCaret$Star_type ==predBag)
```

```
## [1] 0.8333333
```

Caret Bagging

Build Model

```
set.seed(252)
caretBag <- train(Star_type ~.,
                  data = bag.trainCaret,
                  method = "treebag",
                  trControl = trainControl("cv", number = 10),
                  importance = TRUE
)
```

Confusion Matrix

```
table(predCaretBag, bag.testCaret$Star_type)
```

```
##
## predCaretBag  0  1  2  3  4  5
##            0  9  2  0  0  0  0
##            1  3 10  0  0  0  0
##            2  0  0 12  0  0  0
##            3  0  0  0 12  1  0
##            4  0  0  0  0  9  3
##            5  0  0  0  0  2  9
```

Correct Rate

```
mean(predCaretBag == bag.testCaret$Star_type)
```

```
## [1] 0.8472222
```

Boosting

```
?train
```

Model

```
caretBoost <- train(Star_type ~.,
                data = bag.trainCaret,
                method="xgbTree",
                trControl = trainControl("cv", number = 10),
                verbosity = 0)
```

Best Tuning Parameter

```
##    nrounds max_depth eta gamma colsample_bytree min_child_weight subsample
## 60     150         1 0.4     0              0.6                1      0.75
```

Confusion Matrix

```
table(predCaretBoost, bag.testCaret$Star_type)
```

```
##
## predCaretBoost  0  1  2  3  4  5
##              0 11  2  0  0  0  0
##              1  1 10  0  0  0  0
##              2  0  0 12  0  0  0
##              3  0  0  0 12  0  0
##              4  0  0  0  0 10  6
##              5  0  0  0  0  2  6
```

Correct Rate

```
mean(predCaretBoost==bag.testCaret$Star_type)
```

```
## [1] 0.8472222
```

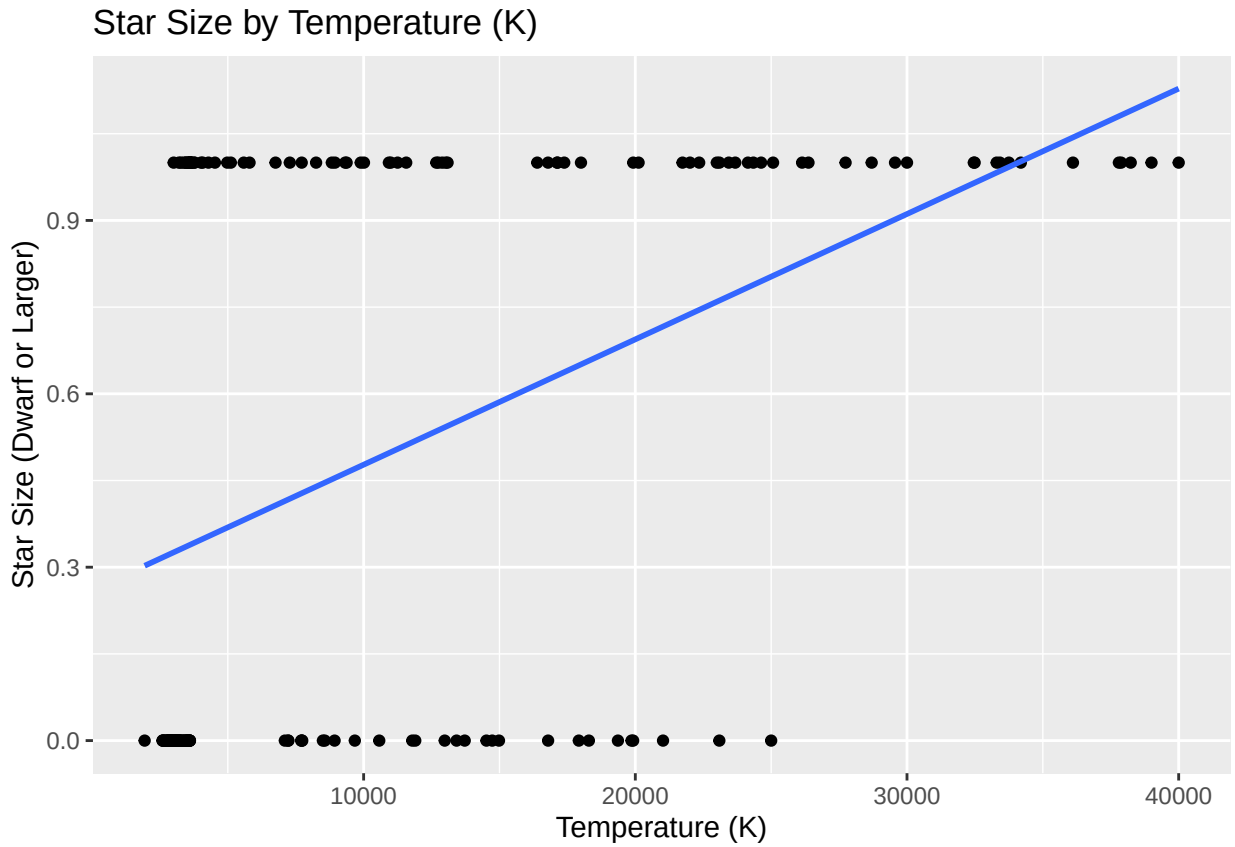
Regression

Variables Used

```
## 'data.frame':    168 obs. of  4 variables:
##  $ Temperature..K. : int  3068 3042 2600 1939 2840 2600 2700 3600 3129 3134 ...
##  $ Luminosity.L.Lo.: num  0.0024 0.0005 0.0003 0.000138 0.00065 0.0004 0.00018 0.0029 0.0122 0.0004
##  $ Star.color      : Factor w/ 6 levels "Blue","Blue white",..: 4 4 4 4 4 4 4 4 4 4 ...
##  $ starSize        : num  0 0 0 0 0 0 0 0 0 0 ...
```

Simple Linear Regression

Plot

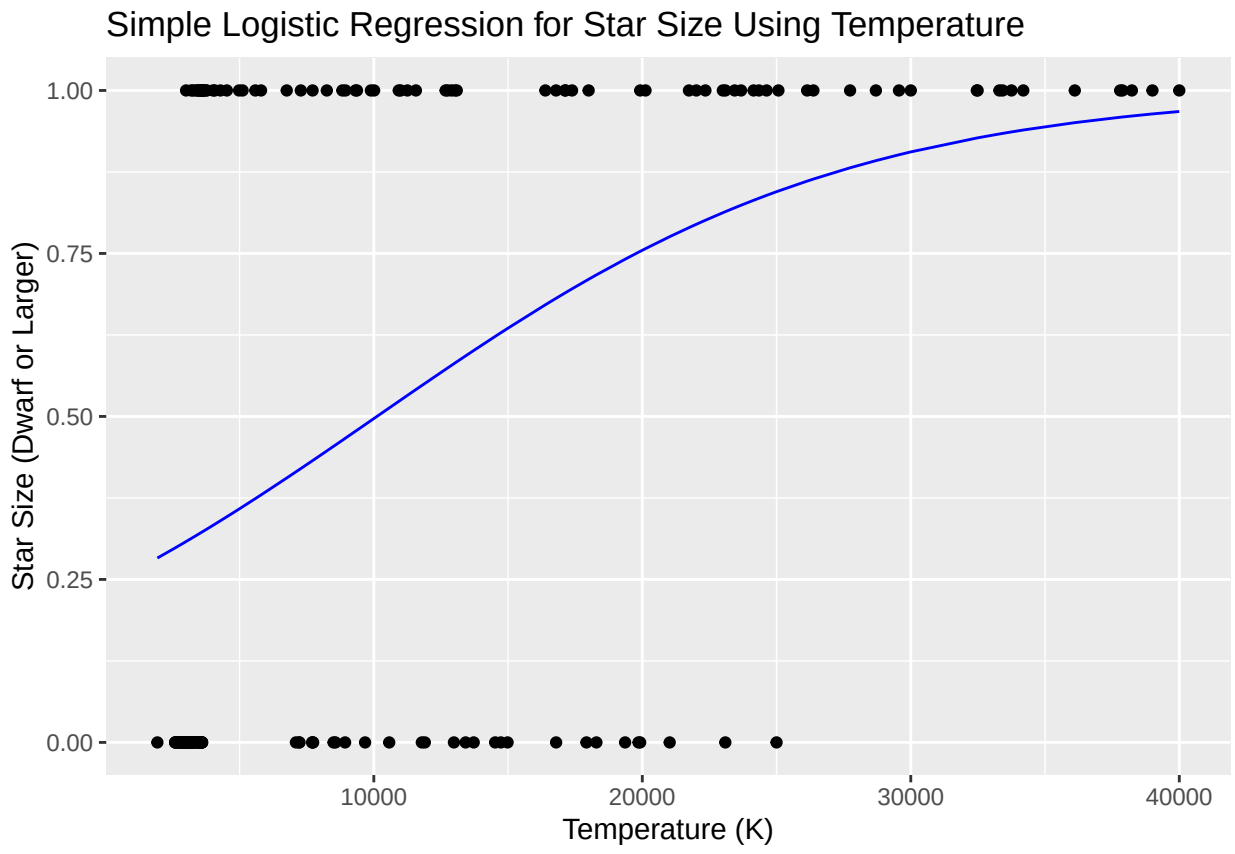


Logistic Regression

Model

```
##
## Call:
## glm(formula = starSize ~ Temperature..K., family = "binomial",
##     data = log.trainCaret)
##
## Coefficients:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)     -1.151e+00  2.635e-01  -4.369 1.25e-05 ***
## Temperature..K.  1.138e-04  2.263e-05   5.030 4.92e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 232.90  on 167  degrees of freedom
## Residual deviance: 195.97  on 166  degrees of freedom
## AIC: 199.97
##
## Number of Fisher Scoring iterations: 4
```

Plot



Confusion Matrix

```
table(conf_matSimp$modSimp, conf_matSimp$testOutcome)
```

```
##
##           0  1
##   FALSE 25 22
##   TRUE  11 14
```

Correct Rate

```
mean(conf_matSimp$modSimp==conf_matSimp$testOutcome)
```

```
## [1] 0.5416667
```

Optimal Logistic Regression Model

Variable Selection

```
modMulti <- glm(starSize ~ .,
                data = log.trainCaret,
                family = "binomial")

summary(modMulti)
```

```
##
## Call:
## glm(formula = starSize ~ ., family = "binomial", data = log.trainCaret)
##
## Coefficients:
##                         Estimate Std. Error z value Pr(>|z|)
## (Intercept)          -2.505e+05  3.329e+05  -0.753    0.452
## Temperature..K.      -1.054e-02  1.979e-02  -0.532    0.594
## Luminosity.L.Lo.      1.975e+00  2.611e+00   0.756    0.450
## Star.colorBlue white  2.506e+05  3.329e+05   0.753    0.452
## Star.colorOrange      2.506e+05  3.330e+05   0.753    0.452
## Star.colorRed         1.045e+05  4.210e+05   0.248    0.804
## Star.colorWhite       2.506e+05  3.667e+05   0.683    0.494
## Star.colorYellowish   2.506e+05  3.330e+05   0.753    0.452
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 232.89745  on 167  degrees of freedom
## Residual deviance:   0.01063  on 160  degrees of freedom
## AIC: 16.011
##
## Number of Fisher Scoring iterations: 25
```

Backward Selection

```
step(modMulti)
```

```
## Start:  AIC=16.01
## starSize ~ Temperature..K. + Luminosity.L.Lo. + Star.color

##                     Df Deviance     AIC
## - Star.color         5    3.615   9.615
## <none>                    0.011  16.011
## - Temperature..K.    1    2.062  16.062
## - Luminosity.L.Lo.   1  179.673 193.673

##
## Step:  AIC=9.61
## starSize ~ Temperature..K. + Luminosity.L.Lo.

##                     Df Deviance     AIC
## - Temperature..K.    1    3.691   7.691
## <none>                    3.615   9.615
## - Luminosity.L.Lo.   1  195.968 199.968

##
## Step:  AIC=7.69
## starSize ~ Luminosity.L.Lo.
##
##                     Df Deviance     AIC
## <none>                    3.691   7.691
## - Luminosity.L.Lo.   1  232.897 234.897

##
## Call:  glm(formula = starSize ~ Luminosity.L.Lo., family = "binomial",
##     data = log.trainCaret)
##
## Coefficients:
##      (Intercept)  Luminosity.L.Lo.
##            -5.05             51.48
##
## Degrees of Freedom: 167 Total (i.e. Null);  166 Residual
## Null Deviance:       232.9
## Residual Deviance: 3.691     AIC: 7.691
```

Forward Selection

```
step(mod0, scope = list(upper = modMulti, lower = mod0),
     method = "forward")
```

```
## Start:  AIC=234.9
## starSize ~ 1

##                     Df Deviance     AIC
## + Luminosity.L.Lo.  1    3.691   7.691
## + Star.color        5  184.500 196.500
## + Temperature..K.   1  195.968 199.968
## <none>                 232.897 234.897

##
## Step:  AIC=7.69
## starSize ~ Luminosity.L.Lo.

##                     Df Deviance     AIC
## <none>                    3.691   7.691
## + Temperature..K.   1    3.615   9.615
## + Star.color        5    2.062  16.062
## - Luminosity.L.Lo.  1  232.897 234.897

##
## Call:  glm(formula = starSize ~ Luminosity.L.Lo., family = binomial,
##     data = log.trainCaret)
##
## Coefficients:
##      (Intercept)  Luminosity.L.Lo.
##            -5.05             51.48
##
## Degrees of Freedom: 167 Total (i.e. Null);  166 Residual
## Null Deviance:       232.9
## Residual Deviance: 3.691     AIC: 7.691
```

Best Subset

```
##   Temperature..K. Luminosity.L.Lo.  Criterion
## 1           FALSE             TRUE   5.691437
## 2            TRUE             TRUE   7.614876
## 3            TRUE            FALSE 197.967986
```

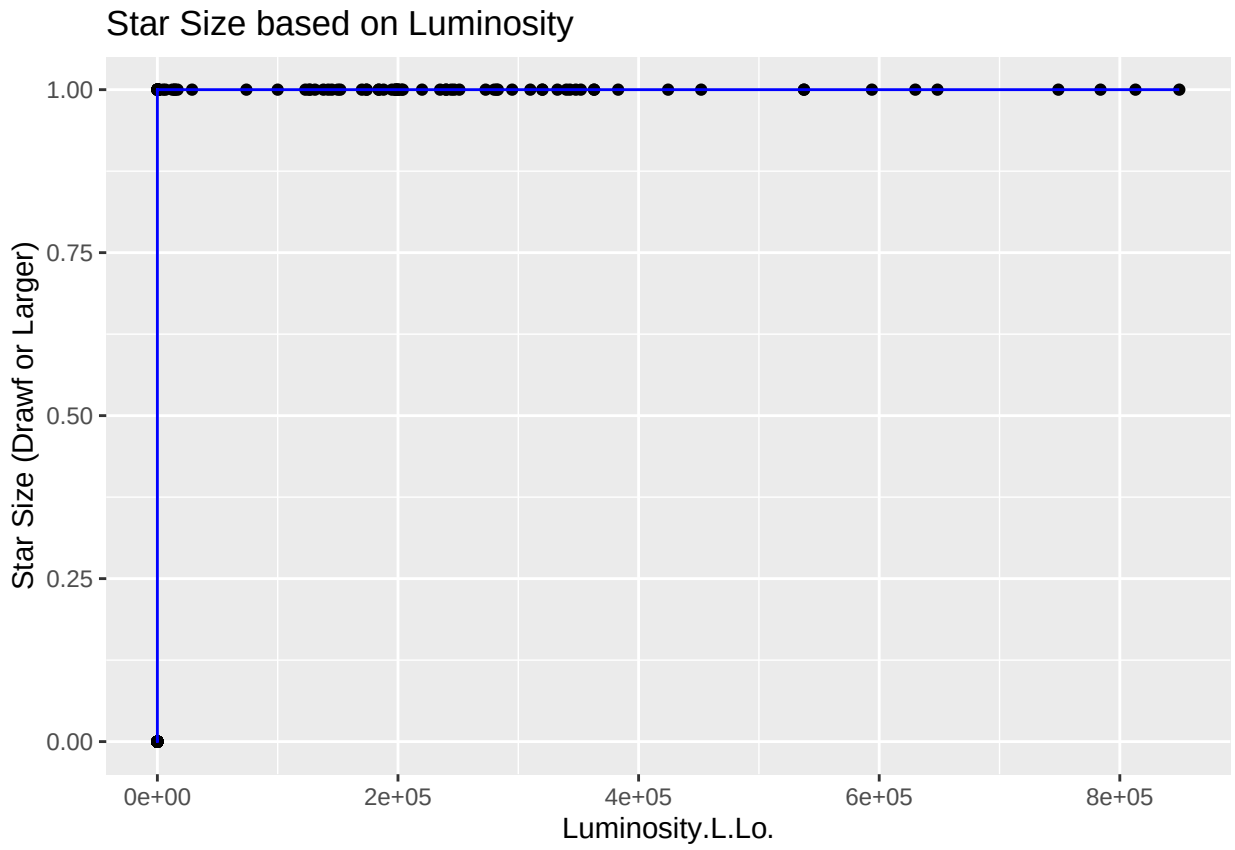
Best Model

```
## Luminosity alone, after removing absolute magnitude and radius, which predicted perfectly
modBest <- glm(starSize ~ Luminosity.L.Lo.,
               data = dplyr::select(log.trainCaret, -Star.color),
               family = "binomial")

summary(modBest)
```

```
##
## Call:
## glm(formula = starSize ~ Luminosity.L.Lo., family = "binomial",
##     data = dplyr::select(log.trainCaret, -Star.color))
##
## Coefficients:
##                  Estimate Std. Error z value Pr(>|z|)
## (Intercept)       -5.0497     0.9484  -5.325 1.01e-07 ***
## Luminosity.L.Lo.  51.4795    12.5818   4.092 4.28e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 232.8975  on 167  degrees of freedom
## Residual deviance:   3.6914  on 166  degrees of freedom
## AIC: 7.6914
##
## Number of Fisher Scoring iterations: 25
```

Plot Best Model



Predict w/ Best Model

Confusion Matrix

```
table(conf_matBest$predOut, conf_matBest$testOutcome)
```

```
##
##           0  1
##   FALSE 36  0
##   TRUE   0 36
```

Correct Rate

```
mean(conf_matBest$predOut==conf_matBest$testOutcome)
```

```
## [1] 1
```

Compare Models

Simple Logistic

```
table(conf_matSimp$modSimp, conf_matSimp$testOutcome)
```

Confusion Matrix

```
##
##          0  1
##   FALSE 25 22
##   TRUE  11 14
```

```
mean(conf_matSimp$modSimp==conf_matSimp$testOutcome)
```

Correct Rate

```
## [1] 0.5416667
```

Optimal Logistic Model

```
table(conf_matBest$predOut, conf_matBest$testOutcome)
```

Confusion Matrix

```
##
##          0  1
##   FALSE 36  0
##   TRUE   0 36
```

```
mean(conf_matBest$predOut==conf_matBest$testOutcome)
```

Correct Rate

```
## [1] 1
```

Discriminant Analysis

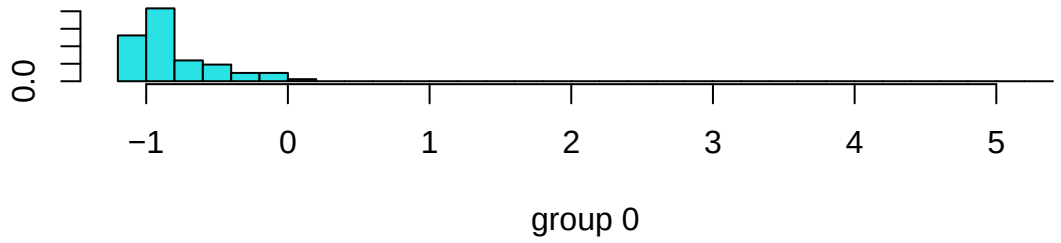
LDA

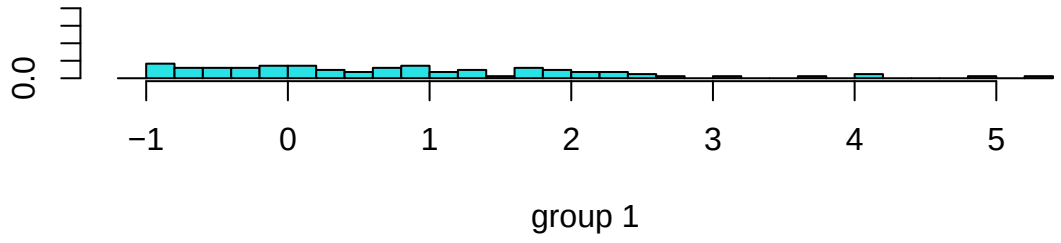
Make Model

```
## Call:
## lda(starSize ~ Temperature..K. + Luminosity.L.Lo., data = da.trainCaret)
##
## Prior probabilities of groups:
##   0   1
## 0.5 0.5
##
## Group means:
##   Temperature..K. Luminosity.L.Lo.
## 0        6586.298     2.525679e-03
## 1       15499.679     2.174372e+05
##
## Coefficients of linear discriminants:
##                             LD1
## Temperature..K.  4.842950e-05
## Luminosity.L.Lo. 5.605833e-06
```

Plot

```
plot(lda)
```





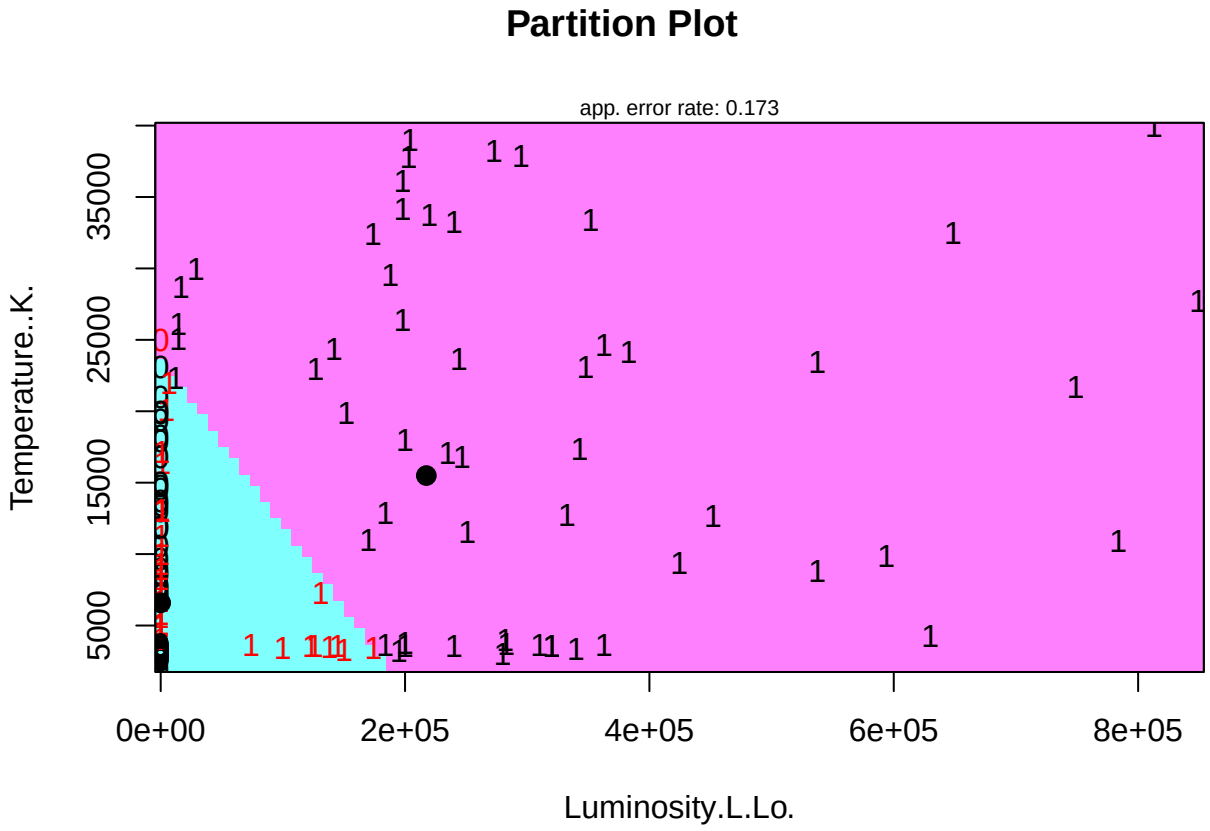
Correct Rate

```
mean(predLDA$class==da.testCaret$starSize)
```

```
## [1] 0.827381
```

Partition Plot

```
partimat(starSize~., data = da.trainCaret, method = "lda")
```



QDA

Make Model

```
## Call:
## qda(starSize ~ Temperature..K. + Luminosity.L.Lo., data = da.trainCaret)
##
## Prior probabilities of groups:
##   0   1
## 0.5 0.5
##
## Group means:
##   Temperature..K. Luminosity.L.Lo.
## 0        6586.298     2.525679e-03
## 1       15499.679     2.174372e+05
```

Correct Rate

```
mean(predQDA$class==da.testCaret$starSize)
```

```
## [1] 0.9940476
```

Partition Plot

```
partimat(starSize~., data = da.trainCaret, method = "qda")
```

